

Reporte de Agentic Loop

0. SKILLS CONFIGURADOS

arquitecto !' DISEÑO-ARQUITECTURA (GLX-DISEÑO-ARQUITECTURA-V1)
tech_lead !' MICROPLANIFICACION-DISEÑO (GLX-MICROPLANIFICACION-DISEÑO-V1)
desarrollador !' FRONTEND-DESARROLLO (GLX-FRONTEND-DESARROLLO-V1)
qa !' QA-VALIDACION (GLX-QA-VALIDACION-V1)

1. ANÁLISIS DE ARQUITECTURA

Micro-ADR: Juego Arcade de Naves Espaciales (PROYEC-21)

1. CONTEXTO

Se requiere un juego web completo de naves espaciales con estética arcade 80s, implementado en HTML5/CSS/JS puro. El juego debe ejecutarse como archivo estático (sin servidor), lo que activa la restricción crítica de entorno LOCAL del skill. El requerimiento especifica una estructura de carpetas con múltiples archivos JS separados, pero el criterio de aceptación de entorno LOCAL prohíbe el uso de ES6 modules (import/export) y scripts de tipo 'module', ya que el protocolo file:/// bloquea CORS y rompe la carga modular.

2. DECISIÓN ARQUITECTÓNICA

Monolito de Archivo Único con Namespacing por Objeto Global (Patrón IIFE/Namespace).

Toda la lógica JavaScript se consolida en un único archivo `js/game.js` organizado internamente mediante el patrón de Módulo Revelador (Revealing Module Pattern) usando objetos literales como namespaces: `Game.Controls`, `Game.Obstacles`, `Game.PowerUps`, `Game.Renderer`, `Game.State`. Esto respeta la separación lógica solicitada en la arquitectura del requerimiento SIN romper la restricción de entorno local.

El canvas HTML5 (`<canvas>`) es el motor de renderizado principal. Se usa el Game Loop clásico con `requestAnimationFrame` para garantizar fluidez. Los assets gráficos se generan proceduralmente mediante Canvas 2D API (formas geométricas, gradientes, efectos de partículas) eliminando dependencia de archivos de imagen externos, lo que cumple la restricción de NO incluir binarios en la lista de archivos.

3. JUSTIFICACIÓN BASADA EN YAGNI

- **¿Por qué NO múltiples archivos JS con módulos ES6?** El entorno de ejecución es file:/// (HTML estático sin servidor). Los navegadores modernos bloquean import/export en este protocolo por política CORS. Dividir en game.js + controls.js + obstacles.js causaría errores silenciosos o pantalla en blanco en el 100% de los casos de uso local. YAGNI: no necesitamos la separación física de archivos si la separación lógica interna cumple el mismo objetivo de mantenibilidad.
- **¿Por qué NO un bundler (Webpack/Vite)?** Introduce una cadena de build que el requerimiento no solicita y que el usuario final no puede ejecutar sin Node.js. KISS: el archivo único funciona con doble-clíc.
- **¿Por qué NO assets de imagen externos?** Generación procedural con Canvas API elimina dependencias de archivos .png/.webp, cumple la restricción del skill y garantiza que el juego funcione offline sin rutas rotas.
- **¿Por qué NO Web Audio API compleja?** Se implementan efectos de sonido sintéticos básicos con OscillatorNode para mantener el stack en 0 dependencias externas.
- **¿Por qué Canvas y no DOM/SVG?** Para un juego con 50+ entidades en pantalla simultáneas, Canvas 2D tiene rendimiento superior al DOM manipulation. SVG no es adecuado para game loops de alta frecuencia.

4. RIESGOS ASUMIDOS POR SIMPLIFICAR

- **Riesgo 1 (BAJO):** El archivo game.js será extenso (~600-900 líneas). Mitigación: comentarios de sección claros y namespacing interno hacen el código navegable.
- **Riesgo 2 (BAJO):** Sin sistema de build, no hay minificación automática. Aceptable para un MVP/juego de entretenimiento.
- **Riesgo 3 (MEDIO):** Los gráficos procedurales tienen menor fidelidad visual que sprites. Mitigación: el estilo arcade 80s (píxeles, neón, formas geométricas) es perfectamente representable con Canvas 2D y es coherente con el requerimiento estético.

- **Riesgo 4 (BAJO):** Sin persistencia de high scores entre sesiones sin localStorage. Mitigación: se implementa localStorage nativo del browser, sin dependencias externas.

5. STACK TECNOLÓGICO FINAL

- **Renderizado:** HTML5 Canvas 2D API
- **Lógica:** JavaScript ES5/ES6 compatible (sin módulos), patrón Namespace
- **Estilos:** CSS3 puro (fuente pixel art via Google Fonts CDN - Press Start 2P)
- **Audio:** Web Audio API (OscillatorNode sintético)
- **Persistencia:** localStorage (high scores)
- **Dependencias externas:** 0 (cero)
- **Infraestructura:** 0 componentes externos (cumple restricción máximo 2)

6. COMPONENTES DEL JUEGO

- **Game.State:** Máquina de estados (MENU !' PLAYING !' PAUSED !' GAME_OVER)
- **Game.Controls:** Event listeners para ArrowKeys, gestión de estado de teclas presionadas
- **Game.Ship:** Física de la nave (inercia, velocidad mínima no-cero, rotación)
- **Game.Obstacles:** Fábrica de 5 tipos (Cometa, Asteroide, Planeta, OVNI, Nebulosa-trampa)
- **Game.PowerUps:** 3 tipos (Velocidad+, Escudo, Imán de puntos)
- **Game.Renderer:** Funciones de dibujo procedural para cada entidad
- **Game.Particles:** Sistema de partículas para explosiones y estela de nave
- **Game.Levels:** Configuración de 3 niveles de dificultad progresiva
- **Game.Score:** Sistema de puntuación por tiempo + objetos recolectados + localStorage

2. MÉTRICAS DE ITERACIÓN (QA)

- index.html | Estado: APROBADO | Iteraciones: 2
- js/game.js | Estado: APROBADO | Iteraciones: 1
- css/styles.css | Estado: APROBADO | Iteraciones: 1
- README.md | Estado: SKIPPED (doc) | Iteraciones: 0

3. MÉTRICAS DE TOKENS IA

Tokens de entrada (input): 90,991

Tokens de salida (output): 26,785

Total tokens: 117,776

Modelo utilizado: claude-sonnet-4-6

Ø=Üh Ø=Ü» ~15.6 horas-hombre ahorradas

"H squad de 4 personas × 2 días

Desglose por Agente:

DISEÑO-ARQUITECTURA: !"2,517 input | !"1,728 output | 1 llamadas | ~1.8h ahorradas

MICROPLANIFICACION-DISEÑO: !"15,096 input | !"8,700 output | 4 llamadas | ~5.8h ahorradas

FRONTEND-DESARROLLO: !"25,583 input | !"11,485 output | 4 llamadas | ~3.9h ahorradas

QA-VALIDACION: !"47,795 input | !"4,872 output | 5 llamadas | ~4.2h ahorradas